

Noise Model Simulator - Iris (3 Klassen, 2 Features)

Modelle: Quantum Kernel SVM + VQC (RealAmplitudes)

Datensatz: Iris (3 Klassen, 2 Features)

Backend: AerSimulator + IBM Noise Model (ibm_marrakesh)

Referenz: `02_iris_fair.ipynb` (Ideal Simulator)

Simuliert Hardware-Rauschen lokal - kein QPU-Kontingent verbraucht.

Zeigt den Effekt von Dekohärenz und Gate-Fehlern auf die Klassifikationsqualität.

0. Imports

```
In [1]: import sys, os
sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.svm import SVC

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.algorithms import VQC
from qiskit_machine_learning.optimizers import SPSA
from qiskit_aer import AerSimulator
from qiskit_aer.noise import NoiseModel
from qiskit_aer.primitives import SamplerV2
from qiskit_ibm_runtime import QiskitRuntimeService

from utils import save_result

print("Imports OK")
```

Imports OK

1. Noise Model laden

```
In [2]: # IBM Account verbinden
service = QiskitRuntimeService()

# Noise Model von ibm_marrakesh laden
backend = service.backend("ibm_marrakesh")
noise_model = NoiseModel.from_backend(backend)

# AerSimulator mit Noise Model
noisy_simulator = AerSimulator(noise_model=noise_model)

print(f"Noise Model geladen von: {backend.name}")
print(f"Basis-Gates: {noise_model.basis_gates}")
```

qiskit_runtime_service.__init__:WARNING:2026-04-30 21:43:45,524: Instance was not set at service instantiation. Free and trial plan instances will be prioritized. Based on the following filters: (tags: None, region: us-east, eu-de), and available plans: (open), the available account instances are: open-instance. If you need a specific instance set it explicitly either by using a saved account with a saved default instance or passing it in directly to QiskitRuntimeService().

qiskit_runtime_service.backends:WARNING:2026-04-30 21:43:45,526: Using instance: open-instance, plan: open

Noise Model geladen von: ibm_marrakesh

Basis-Gates: ['cz', 'delay', 'id', 'if_else', 'measure', 'reset', 'rz', 'sx', 'x']

2. Datensatz - identische Pipeline wie 02_iris_fair

```
In [3]: iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")
```

Train: (105, 2) | Test: (45, 2)

3. Quantum Kernel SVM (Noise Model)

```
In [7]: feature_map_qk = zz_feature_map(feature_dimension=2, reps=2)

# FidelityQuantumKernel mit Noisy Simulator
kernel = FidelityQuantumKernel(feature_map=feature_map_qk)
```

```

# Noisy Sampler
from qiskit_aer.primitives import Sampler as NoisySampler
noisy_sampler = SamplerV2.from_backend(noisy_simulator)
kernel = FidelityQuantumKernel(feature_map=feature_map_qk, fidelity=None)

start = time.time()
svc_q = SVC(kernel=kernel.evaluate)
svc_q.fit(X_train_sc, y_train)
train_time_qk = round(time.time() - start, 4)

start = time.time()
y_pred_qk = svc_q.predict(X_test_sc)
infer_time_qk = round(time.time() - start, 4)

acc_qk = accuracy_score(y_test, y_pred_qk)
f1_qk = f1_score(y_test, y_pred_qk, average="weighted")

print(f"Accuracy: {acc_qk:.4f} | F1: {f1_qk:.4f}")
print(f"Training: {train_time_qk}s | Inferenz: {infer_time_qk}s")
print()
print(classification_report(y_test, y_pred_qk, target_names=iris.target_name

```

```

Accuracy: 0.6222 | F1: 0.6160
Training: 28.2971s | Inferenz: 24.354s

```

	precision	recall	f1-score	support
setosa	0.67	0.80	0.73	15
versicolor	0.64	0.60	0.62	15
virginica	0.54	0.47	0.50	15
accuracy			0.62	45
macro avg	0.62	0.62	0.62	45
weighted avg	0.62	0.62	0.62	45

4. VQC (Noise Model)

```

In [9]: feature_map_vqc = zz_feature_map(feature_dimension=2, reps=1)
        ansatz = real_amplitudes(num_qubits=2, reps=2)

# VQC mit Noisy AerSampler
noisy_sampler_vqc = SamplerV2.from_backend(noisy_simulator)

vqc = VQC(
    feature_map=feature_map_vqc,
    ansatz=ansatz,
    optimizer=SPSA(maxiter=150),
    sampler=noisy_sampler_vqc,
)

print("Starte VQC Training (Noise Model)...")
start = time.time()
vqc.fit(X_train_sc, y_train)
train_time_vqc = round(time.time() - start, 4)

```

```

start = time.time()
y_pred_vqc = vqc.predict(X_test_sc)
infer_time_vqc = round(time.time() - start, 4)

acc_vqc = accuracy_score(y_test, y_pred_vqc)
f1_vqc = f1_score(y_test, y_pred_vqc, average="weighted")

print(f"Accuracy: {acc_vqc:.4f} | F1: {f1_vqc:.4f}")
print(f"Training: {train_time_vqc}s | Inferenz: {infer_time_vqc}s")
print()
print(classification_report(y_test, y_pred_vqc, target_names=iris.target_names))

```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Starte VQC Training (Noise Model)...

Accuracy: 0.5778 | F1: 0.4617

Training: 3366.2989s | Inferenz: 8.4769s

	precision	recall	f1-score	support
setosa	0.65	0.73	0.69	15
versicolor	0.54	1.00	0.70	15
virginica	0.00	0.00	0.00	15
accuracy			0.58	45
macro avg	0.39	0.58	0.46	45
weighted avg	0.39	0.58	0.46	45

5. Ergebnisse speichern

```

In [10]: DATENSATZ = "Iris (3 Klassen, 2 Features, noise)"

save_result("Quantum Kernel SVM (Noise)", DATENSATZ, "AerSimulator+Noise",
save_result("VQC (RealAmplitudes, Noise)", DATENSATZ, "AerSimulator+Noise",

print("Gespeichert.")

```

Backend	Accuracy	Modell		Trainingszeit_s	Inferenzzeit_s	Datensatz	Feature Map	Rep
		Quantum Kernel SVM				Iris (3 Klassen, 2 Features, fair)		
AerSimulator 2.0	0.6222	0,616		28.2243	23,517	zz_feature_map		
		VQC (RealAmplitudes)				Iris (3 Klassen, 2 Features, fair)		
AerSimulator 1.0	0.5333	0.4222		261.5463	0.3237	zz_feature_map		
		Klassischer SVM (RBF)				Iris (3 Klassen, 2 Features, fair)		
lokal	0.9556	0.9556		0.0114	0.0012	NaN	NaN	
		MLP				Iris (3 Klassen, 2 Features, fair)		
lokal	0.9333	0.9327		0.3173	0.0012	(64, 32)	NaN	
		Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 2.0	0.8772	0.9041		408.9585	352.8897	zz_feature_map		
		VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 1.0	0.8713	0.9027		1004.3609	1.2259	zz_feature_map		
		Klassischer SVM (RBF) Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9181	0.9381		0.0076	0.0018	NaN	NaN	
		MLP Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9240	0.9412		0.9143	0.001	(64, 32)	NaN	
		Quantum Kernel SVM (Noise) Iris (3 Klassen, 2 Features, noise)						
AerSimulator+Noise 2.0	0.6222	0.616		28.2971	24.354	zz_feature_map		

Backend	Accuracy	Modell		Trainingszeit_s	Inferenzzeit_s	Datensatz	Feature Map	Rep
		Quantum Kernel SVM				Iris (3 Klassen, 2 Features, fair)		
AerSimulator 2.0	0.6222	0,616		28.2243	23,517	zz_feature_map		
		VQC (RealAmplitudes)				Iris (3 Klassen, 2 Features, fair)		
AerSimulator 1.0	0.5333	0.4222		261.5463	0.3237	zz_feature_map		
		Klassischer SVM (RBF)				Iris (3 Klassen, 2 Features, fair)		
lokal	0.9556	0.9556		0.0114	0.0012	NaN	NaN	
		MLP				Iris (3 Klassen, 2 Features, fair)		
lokal	0.9333	0.9327		0.3173	0.0012	(64, 32)	NaN	
		Quantum Kernel SVM Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 2.0	0.8772	0.9041		408.9585	352.8897	zz_feature_map		
		VQC (RealAmplitudes) Breast Cancer (2 Klassen, 2 Features, fair)						
AerSimulator 1.0	0.8713	0.9027		1004.3609	1.2259	zz_feature_map		
		Klassischer SVM (RBF) Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9181	0.9381		0.0076	0.0018	NaN	NaN	
		MLP Breast Cancer (2 Klassen, 2 Features, fair)						
lokal	0.9240	0.9412		0.9143	0.001	(64, 32)	NaN	
		Quantum Kernel SVM (Noise) Iris (3 Klassen, 2 Features, noise)						
AerSimulator+Noise 2.0	0.6222	0.616		28.2971	24.354	zz_feature_map		
		VQC (RealAmplitudes, Noise) Iris (3 Klassen, 2 Features, noise)						
AerSimulator+Noise 1.0	0.5778	0.4617		3366.2989	8.4769	zz_feature_map		

Gespeichert.

6. Vergleich: Ideal vs. Noise Model

```
In [3]: print(df.dtypes)
print(df["Accuracy"].head(10))
```

```
Modell          str
Datensatz       str
Backend        str
Accuracy        str
F1             str
Trainingszeit_s float64
Inferenzzeit_s  str
Feature Map     str
Reps           float64
dtype: object
0    0.6222
1    0.5333
2    0.9556
3    0.9333
4    0.8772
5    0.8713
6    0.9181
7    0.924
8    0.6222
9    0.5778
Name: Accuracy, dtype: str
```

```
In [4]: print(df["Accuracy"].unique())
```

```
<StringArray>
['0.6222', '0.5333', '0.9556', '0.9333', '0.8772', '0.8713', '0.9181',
 '0.924', '0.5778', '0.8421', 'COBYLA']
Length: 11, dtype: str
```

```
In [5]: df = pd.read_csv("Ergebnisse/ergebnisse.csv")

df_ideal = df[df["Datensatz"] == "Iris (3 Klassen, 2 Features, fair)"][["Modell", "Accuracy (Ideal)", "F1 (Ideal)"]]
df_ideal.columns = ["Modell", "Accuracy (Ideal)", "F1 (Ideal)"]

df_noise = df[df["Datensatz"] == "Iris (3 Klassen, 2 Features, noise)"][["Modell", "Accuracy (Noise)", "F1 (Noise)"]]
df_noise.columns = ["Modell", "Accuracy (Noise)", "F1 (Noise)"]

# Modellnamen vereinheitlichen für Merge
df_noise["Modell"] = df_noise["Modell"].str.replace(" (Noise)", "", regex=False)

df_cmp = pd.merge(df_ideal, df_noise, on="Modell", how="outer")
df_cmp["Δ Accuracy"] = (df_cmp["Accuracy (Noise)"] - df_cmp["Accuracy (Ideal)"])

print(df_cmp.to_string(index=False))
```

	Modell	Accuracy (Ideal)	F1 (Ideal)	Accuracy (Noise)	F
1 (Noise)	Δ Accuracy				
	Klassischer SVM (RBF)	0.9556	0.9556	NaN	
NaN	NaN				
	MLP	0.9333	0.9327	NaN	
NaN	NaN				
	Quantum Kernel SVM	0.6222	0,616	0.6222	
616	0.0				
	VQC (RealAmplitudes)	0.5333	0.4222	NaN	
NaN	NaN				
	VQC (RealAmplitudes, Noise)	NaN	NaN	0.5778	
0.4617	NaN				